ps1-writer

Technical reference and porting guide

This document describes everything the PlayStation program expects from the Pico, so the Pico firmware can be rewritten without touching the PS1 side. It also explains how the existing firmware is organised, the traps found while building it, and how to test and debug a new version.

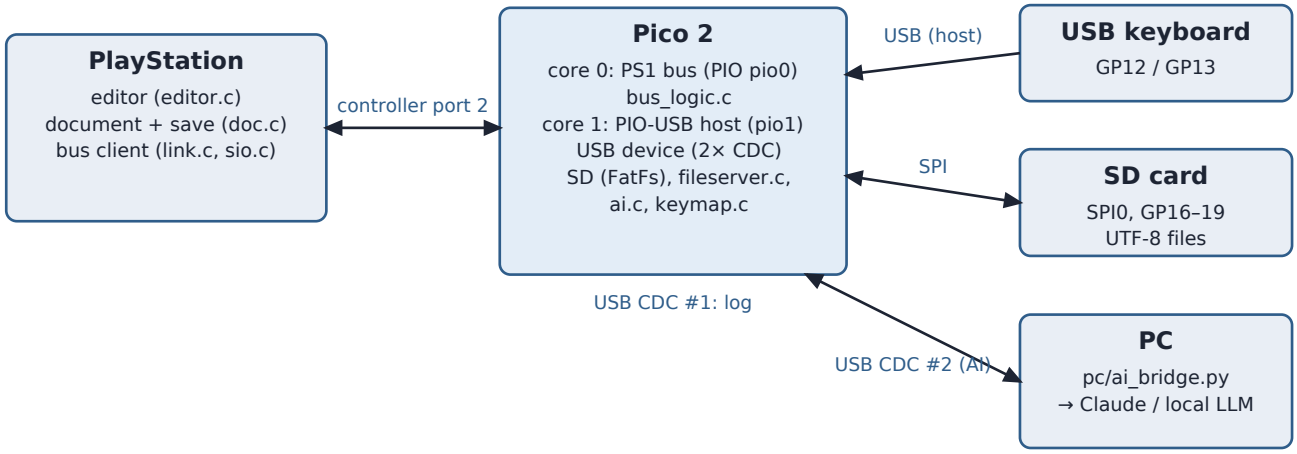
Protocol version: 2 **Device address:** 0x22 **Text encoding:** Latin-1 (ISO-8859-1) on the bus, UTF-8 on the SD card

> **The one rule.** The PS1 only sees bytes on the controller bus. A new Pico firmware that answers those bytes exactly as described in sections 3 to 6 works with the existing PS1 discs, however it is organised inside. `common/pw_protocol.h` is the source of truth; this document explains it.

Contents

1 System overview
2 Hardware and pins
3 Controller bus: electrical and timing
4 Bus transactions: KEYS, REQUEST, RESPONSE
5 Requests and responses
6 Saving: patches, verification and recovery
7 The AI link
8 Inside the current Pico firmware
9 Traps and checklist for a rewrite
10 Reading the PS1 code
11 Testing and debugging

1 System overview



The PS1 is always the bus master. Every frame (60 times a second) it polls the Pico for key events. File work uses a request/response pair: the PS1 sends a request, then keeps asking for the response until the Pico has one. The Pico does slow work (SD card, PC) on its second core, so the bus never waits for it.

Part	Where	Job
PS1 program	`ps1/`	Editor UI, document in RAM as paragraphs, builds save patches, draws 80 columns × 20–30 rows.
Shared contract	`common/`	`pw_protocol.h` (all bytes and constants), `charset.h` (Latin-1 ⇄ UTF-8).
Pico firmware	`pico/`	Bus device, USB keyboard host, SD card file server, AI link, log port.
PC bridge	`pc/ai_bridge.py`	Talks to the Pico's second USB serial port and to an LLM.
Tests	`tests/`	Compile PS1 + Pico logic for the PC; run the bridge against the Pico AI code.

2 Hardware and pins

Signal	Pico GPIO	Pico pin	Notes
PS1 CMD (port pin 2)	GP2	4	input; must be CLK − 1 (PIO reads both with one instruction)
PS1 CLK (port pin 7)	GP3	5	input; idles high
PS1 ATT (port pin 6)	GP4	6	input; low = selected (shared with memory card slot 2)
PS1 DATA (port pin 1)	GP5	7	open drain: only pull low or release
PS1 ACK (port pin 9)	GP6	9	open drain: only pull low or release
PS1 GND (port pin 4)	GND	8	common ground
PS1 7.6 V / 3.5 V (pins 3, 5)	—	—	never connect
USB keyboard D+	GP12	16	PIO-USB; D− must be D+ + 1
USB keyboard D−	GP13	17	optional 22 Ω series, 15 kΩ to GND
USB keyboard 5 V	VBUS	40	from the Pico's own USB cable
SD MISO / CS / SCK / MOSI	GP16 / 17 / 18 / 19	21 / 22 / 24 / 25	SPI0 at 1 MHz (4 MHz failed on long wires)
SD 3.3 V	3V3(OUT)	36	bare card is 3.3 V only
Pico native USB	—	—	two CDC serial ports: #1 log, #2 AI link

System clock 120 MHz (PIO-USB needs a multiple of 12 MHz). The bus state machine runs at 50 MHz (20 ns per instruction).

3 Controller bus: electrical and timing

- SPI-like, **LSB first**, 8-bit bytes. CLK idles high. The device puts its bit on DATA after CLK falls; both sides sample on the rising edge (SPI mode 3).
- Bus clock is set by the disc: 250 kHz (stock), 500 kHz or 1 MHz (`SIO_BAUD_RELOAD` 0x88 / 0x44 / 0x22).
- DATA and ACK are open drain with pull-ups in the console. The Pico never drives them high; it switches the pin direction (output = pull low). That is why the reply byte is **inverted** before it goes into the PIO FIFO.
- After each byte the device pulses ACK low. The current firmware waits about 3.2 µs, then holds ACK low for about 3.2 µs.
- **The PS1 requires an ACK after every byte of a ps1-writer transaction, including the last one.** `ps1/sio.c` polls for the ACK (3000 loop iterations) and `link.c` treats a missing ACK as a bus error. A normal controller does not ACK its last byte, so code copied from controller emulators fails here.
- The reply is one byte behind: the byte the Pico sends while receiving byte *k* was chosen after byte *k−1*. During the address byte the Pico sends 0xFF.

> Core 0 must react between bytes within microseconds. Run the bus loop and everything it calls from RAM (`__not_in_flash_func`), never print or touch the SD card there, and check with `objdump` that it makes no calls into flash.

4 Bus transactions

Each transaction starts with ATT low and the address byte 0x22, followed by a command byte. Any other address means the transaction is for a controller or memory card: stay silent (no ACK, DATA released). Byte columns below show what each side sends at the same time.

4.1 KEYS (0x01): poll keyboard events, every frame

Byte	PS1 sends	Pico sends	Meaning
0	22	FF	address
1	01	FF	command KEYS
2	00	flags	bit0 keyboard, bit1 SD card, bit2 AI bridge
3	00	n	number of events, 0–16
4…	00	kind code mods	3 bytes per event, n events

- Remove events from the queue **only after the PS1 has clocked out the last byte**. If the transfer is cut short, the same events are sent again next frame: no key is lost or doubled.
- Events: kind 1 = character (code is Latin-1), kind 2 = key (code below), kind 3 = Ctrl+letter (code is the lowercase letter). mods: 0x01 Ctrl, 0x02 Shift, 0x04 Alt, 0x08 AltGr.
- Key codes: UP 1, DOWN 2, LEFT 3, RIGHT 4, HOME 5, END 6, PGUP 7, PGDN 8, BACKSPACE 9, DELETE 10, ENTER 11, TAB 12, ESC 13, INSERT 14, F1–F12 = 0x40–0x4B.
- **Key repeat is the Pico's job** (500 ms delay, then every 33 ms). The PS1 has none.
- KEYS has no CRC. Key data is short and the bus has proven reliable.

4.2 REQUEST (0x10): send a request

Byte	PS1 sends	Pico sends	Meaning
0	22	FF	address
1	10	FF	command REQUEST
2	lenL	status	A5 = ready, carry on; anything else = busy, PS1 stops and retries
3	lenH	FF	payload length (≤ 2048)
4…	payload	FF	[op][seq][arguments]
end	crcL crcH	FF	CRC-16 over lenL, lenH and payload

- Accept a new request only when the previous one has been answered (mailbox idle). Otherwise send busy (0x01) as the status and stop ACKing after that byte.
- If the CRC is wrong, silently drop the request. The PS1 notices because no response with its sequence number appears, and it sends the request again.

4.3 RESPONSE (0x11): fetch the answer

Byte	PS1 sends	Pico sends	Meaning
0	22	FF	address
1	11	FF	command RESPONSE
2	00	status	A5 = answer follows · 01 = still working · 00 = no request pending
3, 4	00 00	lenL lenH	only if status A5
5…	00…	payload	[op][seq][result][data]
end	00 00	crcL crcH	CRC-16 over lenL, lenH and payload

- Keep the last answer available until the next request is accepted, so the PS1 can read it again after a CRC error.
- CRC-16/CCITT-FALSE: polynomial 0x1021, initial value 0xFFFF, no reflection, no final XOR (`pw_crc16()`).

5 Requests and responses

Request payload: `[op][seq][arguments]`. Response payload: `[op][seq][result][data]`. Numbers are little endian. Always echo op and seq: the PS1 ignores an answer whose sequence number doesn't match and resends its request.

Op	Name	Arguments	Data in the answer	PS1 timeout
01	HELLO	—	[version=2][flags][max payload u16]. Answer even without an SD card.	0.5 s
02	LIST	[start u16]	[total u16][count] then per file [name len][name][size u32]. Sorted; start > 0 continues the list.	3 s
03	OPEN	[name] (empty = last used)	[size u32][name len][name]. Converts the UTF-8 file to the Latin-1 working copy.	120 s
04	READ	[offset u32][len u16]	up to 2000 bytes of the working copy	3 s
05	CREATE	[name]	— (new empty file becomes the open file; EXISTS if taken)	5 s
06	SAVE_BEGIN	—	—	120 s
07	SAVE_DATA	[chunk u16][save ops]	—	120 s
08	SAVE_END	[size u32][crc32 u32]	[size u32]	120 s
09	CONFIG_GET	—	settings bytes (≤ 64, empty if none). Opaque to the Pico.	2 s
0A	CONFIG_SET	[settings bytes ≤ 64]	—	2 s

Op	Name	Arguments	Data in the answer	PS1 timeout
20	AI_ASK	[paragraph u32][offset u32][question ≤ 1000]	— (see section 7)	30 s
21	AI_POLL	—	[state][next piece of the answer ≤ 1900]	3 s
22	AI_CANCEL	—	—	2 s

Results: 0 OK · 1 NOT_FOUND · 2 EXISTS · 3 IO · 4 NO_SD · 5 BAD_ARGS · 6 BAD_STATE · 7 VERIFY · 8 NOT_AVAILABLE · 9 TOO_BIG. Every operation except HELLO and the AI operations answers NO_SD while no card is mounted.

Timeouts. The table shows the total time the PS1 waits. Independently, if the Pico gives *no* answer at all (not even "busy") for 3 s, the PS1 declares it gone. Long jobs are fine as long as RESPONSE keeps answering 0x01 busy.

Names

- File names travel in Latin-1, at most 48 bytes. Forbidden: / \ : * ? " < > | , a leading dot or space. The PS1 appends `.md` when there is no extension.
- Documents live in `/ps1writer/` as UTF-8. Names with characters Latin-1 lacks are listed with stand-ins (ş → s). OPEN must find them again by comparing the Latin-1 form of each directory entry (the current firmware's `open_lookup()`).
- Text conversion on OPEN: drop CR, map typographic characters to ASCII (`unicode_fallback()`), anything else becomes '?'. The PS1 assumes exactly this in its tests.

6 Saving: patches, verification and recovery

The Pico keeps a **working copy**: the Latin-1 text of the open document as last opened or saved. The PS1 knows where each unchanged paragraph sits in it. A save describes the new file as a list of pieces:

Save op	Bytes	Meaning
KEEP (0x01)	`[offset u32][len u32]`	copy this range of the working copy
BYTES (0x02)	`[len u16][bytes]`	new Latin-1 text

```
SAVE_BEGIN                          open working copy, start the new file
SAVE_DATA chunk=0  [KEEP 0,51234][BYTES 37 "new sentence ..."][KEEP 51271,...]
SAVE_DATA chunk=1  ...              at most 1900 bytes of ops per chunk
SAVE_END  size, crc32               Pico compares, then swaps files
```

- Chunks are numbered from 0. A repeat of the previous chunk number is a resend: answer OK and do not apply it again. Any other out-of-order number: abort, BAD_ARGS.
- SAVE_END: compare the total size and the **CRC-32 (IEEE)** of the new Latin-1 text with the PS1's values. Mismatch: answer **VERIFY** and leave the old file alone. The PS1 then sends the whole document as BYTES.
- Only after a match: move the old document to `backup/name.1` (shift .1→.2 … drop .5), put the new UTF-8 file in place, and make the new Latin-1 text the working copy.
- No open file (for example after a Pico reboot): SAVE_BEGIN answers **BAD_STATE** (so do SAVE_DATA / SAVE_END outside a save). The PS1 re-OPENs the document by name and does a full save.
- The PS1 lets the user keep typing while SAVE_DATA / SAVE_END are processed; the Pico does not need to know.

These checks are what make saving safe. Keep VERIFY and BAD_STATE even in a simplified rewrite: without them, a lost or misapplied patch would silently corrupt the thesis.

7 The AI link

PS1 side

- **AI_ASK**: the PS1 has just saved the document. The Pico sends the question, the cursor position and the saved UTF-8 file of the open document to the PC, then answers OK. If the PC bridge has not said HELLO, answer NOT_AVAILABLE.
- **AI_POLL** (every 100 ms): the answer's state and the next piece of text (Latin-1, up to 1900 bytes). States: 0 idle, 1 more to come (the text may be empty), 2 done (this is the last piece), 3 error (the text is the message).
- **AI_CANCEL**: forget the question; tell the PC CANCEL.
- Set flag bit 2 (AI) in KEYS / HELLO while the bridge is connected.

PC side: second USB serial port (CDC #2)

Direction	Line	Followed by
Pico → PC	`ASK <paragraph> <offset> <doc bytes> <question bytes>\n`	the document (UTF-8), then the question (UTF-8)
Pico → PC	`CANCEL\n`	—
PC → Pico	`HELLO <anything>\n`	— (sent on connect and every 5 s)
PC → Pico	`TXT <n>\n`	n bytes of answer text (UTF-8)
PC → Pico	`END\n`	— the answer is complete
PC → Pico	`ERR <n>\n`	n bytes of error message (UTF-8)

- The Pico converts the answer to Latin-1 (same stand-ins as OPEN) and buffers up to 64 KB.
- If the port closes (no DTR) while a question is out, report an error to the PS1.
- `pc/ai_bridge.py` finds the port by USB VID:PID cafe:4021 and the interface name "ps1-writer AI". Keep those if you change the USB descriptors, or use `--port`.

8 Inside the current Pico firmware

File	Runs on	Job
`main.c`	both	clock 120 MHz, starts core 1 with a 16 KB stack, PIO setup, core 0 bus loop, core 1 main loop, USB CDC glue
`sio0_dev.pio`	pio0	bit-level bus device: shifts bytes, drives DATA and ACK by pin direction
`bus_logic.c`	core 0	byte-by-byte state machine for KEYS / REQUEST / RESPONSE (portable C, from RAM)
`shared.c/h`	both	lock-free key queue (core 1 writes, core 0 reads) and the request/response mailbox
`fileserver.c`	core 1	all file operations, conversions, save patching, backups
`vfs.h, vfs_fatfs.c`	core 1	the few file calls the file server needs, on FatFs
`keyboard.c, keymap.c`	core 1	TinyUSB HID callbacks → US layout → events, key repeat
`ai.c/h`	core 1	AI link: PC protocol parser, answer buffer, AI_* operations
`usb_descriptors.c, tusb_config.h`	core 1	composite USB device: two CDC ports; host on PIO-USB

Mailbox states

```
IDLE --(core 0: request with good CRC)--> PENDING --(core 1 takes it)--> PROCESSING
  ^                                                                          |
  +----------------(core 1: response written, resp_valid = 1)--------------+
REQUEST is accepted only in IDLE. RESPONSE answers "busy" unless IDLE.
```

SD card layout

```
/ps1writer/                documents (UTF-8)
/ps1writer/backup/         name.1 (newest) ... name.5
/ps1writer/.work/doc.latin1  working copy      last.txt  last opened name
/ps1writer/.work/doc.new, out.tmp  save in progress   settings.bin  CONFIG bytes
```

9 Traps and checklist for a rewrite

Problems already met while building the current firmware:

Trap	Symptom	Fix
Bus code running from flash	random ACK timeouts	`__not_in_flash_func` on the bus loop and everything it calls; no flash SDK helpers inside (use inline `pio_encode_*` + `pio_sm_exec`)
DMA channel claimed twice	Pico halts right after USB starts; PC sees no serial port	PIO-USB claims `tx_ch` itself: find a free channel, unclaim it, pass the number
Core 1 stack 2 KB (default)	crash when the SD card mounts	`multicore_launch_core1_with_stack()` with ≥ 16 KB
PIO-USB uses all of pio1	`pio_add_program` panics	keep the bus program on pio0
Missing ACK on the last byte	every transfer fails	ACK every byte of our transactions
Forgetting the reply lag	PS1 reads shifted bytes	prepare the next reply after each received byte
Committing keys too early	lost keys after a bus glitch	advance the queue tail only after the last byte
SD clock 4 MHz on long wires	mount or read errors	1 MHz
FatFs rename onto an existing file	backup rotation fails	delete the target first
FatFs without UTF-8 long names	names like Größe.md fail	`FF_USE_LFN` ≥ 1, `FF_LFN_UNICODE` 2
Pico-PIO-USB version	build errors	commit fe9133f matches TinyUSB 0.18 (Pico SDK 2.2)

Known open issues worth fixing in a rewrite

- A keyboard already plugged in when the Pico powers up is not detected.
- One wired mechanical keyboard did not work (a wireless keyboard with a receiver did). The keymap only accepts 8-byte boot reports; a keyboard that ignores the boot protocol and sends a report ID or NKRO bitmap will not work. `kbtest` prints raw reports to check.

Checklist before trying a new firmware on the console

- HELLO answers version 2 and correct flags, with and without an SD card.
- KEYS: 4 bytes when idle, all bytes ACKed, events committed only at the end.
- REQUEST: status A5 on the third byte; bad CRC dropped silently.
- RESPONSE: busy while working, A5 + payload + CRC when done; answer kept for re-reads.
- OPEN "" returns the last document or NOT_FOUND.
- SAVE_END returns VERIFY on a wrong CRC; SAVE_BEGIN returns BAD_STATE without an open file.
- `make test` in `tests/` passes (see section 11).

10 Reading the PS1 code

Order	File	Size	Why read it
1	`common/pw_protocol.h`	~180 lines	the contract; the top comment shows every transaction
2	`ps1/link.c`	~170 lines	the PS1 side of the bus: the mirror image of what the Pico must do (ACK checks, retries, seq matching, the 3 s rule)
3	`ps1/sio.c`	~70 lines	PS1 controller-port registers (same as the video player)
4	`ps1/doc.c`	~520 lines	paragraphs, loading via READ, how save patches are built and verified
5	`ps1/main.c`	~230 lines	screen (sprites from an 8×8 font texture), clock, main loop
6	`ps1/editor.c`	~1300 lines	UI: keys, wrapping, dialogs, AI chat; skip unless changing behaviour

Changing the PS1 side. Needs PSn00bSDK 0.24. In `ps1/`: `cmake --preset default`, `cmake --build build` → `build/ps1writer.bin/.cue`. editor.c, doc.c and link.c are plain C and covered by the PC tests, so most changes can be tried without burning a disc.

11 Testing and debugging

PC tests

`cd tests && make test` compiles the PS1 editor and the Pico's bus logic, file server and AI link for the PC, with a folder as the SD card. A rewrite that keeps these three interfaces gets all 160+ checks for free:

```
uint8_t  bus_begin(bus_t *b);                                  /* bus_logic.h */
int      bus_byte(bus_t *b, uint8_t rx, uint8_t *next_reply);
void     bus_end(bus_t *b);
uint16_t fileserver_process(const uint8_t *req, uint16_t len, uint8_t *resp);
int      vfs_open(...), vfs_read(...), ...                     /* vfs.h */
```

- `tests/sio_host.c` plays the PS1's port driver by calling bus_byte directly, and can flip random bits to test CRCs and retries.
- `tests/test_bridge.py` runs the real Python bridge against `pico/ai.c` over a pseudo-terminal.

On the hardware

- **Log port** (USB CDC #1, any baud rate): a status line every 5 s, plus every open and save.
- **LED**: slow blink = firmware running; flicker = file requests; dark = halted.
- **kbtest.uf2**: keyboard only. Shows electrical detection, enumeration, raw reports and decoded characters.

Logic analyser (Arduino Nano + PulseView)

Nano pin	Signal	Pico pin
D8	ATT (trigger: low)	GP4
D9	CLK	GP3
D10	CMD (MOSI)	GP2
D11	DATA (MISO)	GP5
D12	ACK	GP6
GND	ground	GND

- 1 kΩ in series with every signal; Nano powered from its own USB; never connect its 5 V pin.
- SUMP driver, 2 MHz (or 1 MHz), 1024 samples, 250 kHz disc.
- SPI decoder: CS# = ATT, CLK, MOSI = CMD, MISO = DATA, CPOL 1, CPHA 1, **LSB first**.
- Expected idle poll: `22/FF 01/FF 00/03 00/00`, with an ACK pulse after every byte.
- At 2 MHz the decoder sometimes reads the last bit of a byte as 1 (81, 83 instead of 01, 03): the Pico releases DATA about 60 ns after the clock edge. The PS1 is not affected.